

sentiment.py

```
1 import numpy as np
2 import pandas as pd
3 import re
4 import pickle
5 import nltk
6
7 from nltk.corpus import stopwords
8 from nltk.stem.porter import PorterStemmer
9
10 from sklearn.feature_extraction.text import TfidfVectorizer
11 from sklearn.model_selection import train_test_split
12 from sklearn.linear_model import LogisticRegression
13 from sklearn.metrics import accuracy_score
14
15 # Download stopwords
16 nltk.download('stopwords')
17
18 # Load stopwords
19 stop_words = set(stopwords.words('english'))
20
21 # Load dataset
22 dataset = pd.read_csv(
23     "training.1600000.processed.noemoticon.csv",
24     encoding='ISO-8859-1',
25     header=None
26 )
27
28 # Add column names
29 col_names = ['target', 'id', 'date', 'flag', 'user', 'text']
30 dataset.columns = col_names
31
32 # Show first 5 rows
33 print(dataset.head())
34
```

```
35 # Dataset shape
36 print("Dataset Shape:", dataset.shape)
37
38 # Check missing values
39 print(dataset.isnull().sum())
40
41 # Target distribution
42 print(dataset['target'].value_counts())
43
44 # Convert 4 -> 1 (Positive)
45 dataset['target'] = dataset['target'].map({0: 0, 4: 1})
46
47 print(dataset['target'].value_counts())
48
49 # Initialize stemmer
50 stemmer = PorterStemmer()
51
52 # Stemming function
53 def stemming(content):
54     content = str(content)
55
56     # Remove non-alphabet characters
57     stemmed_content = re.sub('[^a-zA-Z]', ' ', content)
58
59     # Convert to lowercase
60     stemmed_content = stemmed_content.lower()
61
62     # Split words
63     stemmed_content = stemmed_content.split()
64
65     # Remove stopwords and stem
66     stemmed_content = [
67         stemmer.stem(word)
68         for word in stemmed_content
69         if word not in stop_words
70     ]
71
```

```
72     # Join words
73     stemmed_content = ' '.join(stemmed_content)
74
75     return stemmed_content
76
77 # Apply stemming
78 dataset['text'] = dataset['text'].apply(stemming)
79
80 print(dataset.head())
81
82 # Separate data and labels
83 X = dataset['text']
84 Y = dataset['target']
85
86 # Train-test split
87 X_train, X_test, Y_train, Y_test = train_test_split(
88     X,
89     Y,
90     test_size=0.2,
91     random_state=42
92 )
93
94 # Convert text to numerical data
95 vectorizer = TfidfVectorizer()
96
97 X_train = vectorizer.fit_transform(X_train)
98 X_test = vectorizer.transform(X_test)
99
100 print(X_train)
101
102 # Train model
103 model = LogisticRegression(max_iter=1000)
104
105 model.fit(X_train, Y_train)
106
107 # Predict
108 Y_pred = model.predict(X_test)
```

```
109
110 # Accuracy
111 accuracy = accuracy_score(Y_test, Y_pred)
112
113 print("Accuracy:", accuracy)
114
115 # Prediction function
116 def predict_sentiment(text):
117
118     text = re.sub('[^a-zA-Z]', ' ', text)
119
120     text = text.lower()
121
122     text = text.split()
123
124     text = [
125         stemmer.stem(word)
126         for word in text
127         if word not in stop_words
128     ]
129
130     text = ' '.join(text)
131
132     text = [text]
133
134     text = vectorizer.transform(text)
135
136     sentiment = model.predict(text)
137
138     if sentiment[0] == 0:
139         return "Negative"
140     else:
141         return "Positive"
142
143 # Test predictions
144 print(predict_sentiment("I hate you"))
145 print(predict_sentiment("I love you"))
```

```
146
147 # Save model
148 with open('model.pkl', 'wb') as model_file:
149     pickle.dump(model, model_file)
150
151 # Save vectorizer
152 with open('vectorizer.pkl', 'wb') as vectorizer_file:
153     pickle.dump(vectorizer, vectorizer_file)
154
155 print("Model and vectorizer saved successfully!")
```